

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Лабораторная работа 1

По дисциплине Объектно-ориентированное программирование

Тема работы Алгоритмы хеширования

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Иванов С.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 УПРАЖНЕНИЕ 1.....	4
2 УПРАЖНЕНИЕ 2.....	7
3 КОНТРОЛЬНЫЕ ВОПРОСЫ	9
ЗАКЛЮЧЕНИЕ	10

ВВЕДЕНИЕ

В отчёте представлено решение лабораторной работы. В тексте работы представлены результаты выполнения двух упражнений и ответы на контрольные вопросы.

Цель — изучить алгоритмы хеширования, реализовать алгоритм Рабина-Карпа поиска подстроки в строке с применением хеширования.

Задание на лабораторную работу

1. Реализовать поиск одинаковых строк.

Дан список строк $S[1..n]$, каждая длиной не более m символов. Требуется найти все повторяющиеся строки и разделить их на группы, чтобы в каждой группе были только одинаковые строки.

2. Реализовать алгоритм Рабина-Карпа поиска подстроки в строке за $O(n)$.

1 УПРАЖНЕНИЕ 1

Был обеспечен безопасный ввод строк и их количества пользователем, предрасчёт степеней p^i (рисунок 1). Максимальная длина строки m вычисляется автоматически. Количество символов $p = 31$ учитывает строчные латинские буквы и знаки препинания.

```
1      static void Main(string[] args)
2      {
3          int p = 31;
4
5          // Input n - count of words
6          int n = 0;
7          Console.Write("Write n = ");
8          while (!int.TryParse(Console.ReadLine(), out n))
9          {
10             Console.Write("Must be positive int! Write n = ");
11          }
12
13         // Input words
14         Console.WriteLine("Input strings:");
15         string[] strings = new string[n];
16         int m = 0;
17         for (int i = 0; i < n; i++)
18         {
19             strings[i] = Console.ReadLine();
20             if (strings[i].Length > m) m = strings[i].Length;
21         }
22
23         // Calculate powers
24         int[] p_pow = new int[m];
25         p_pow[0] = 1;
26         for (int i = 1; i < m; i++)
27         {
28             p_pow[i] = p_pow[i - 1] * p;
29         }
```

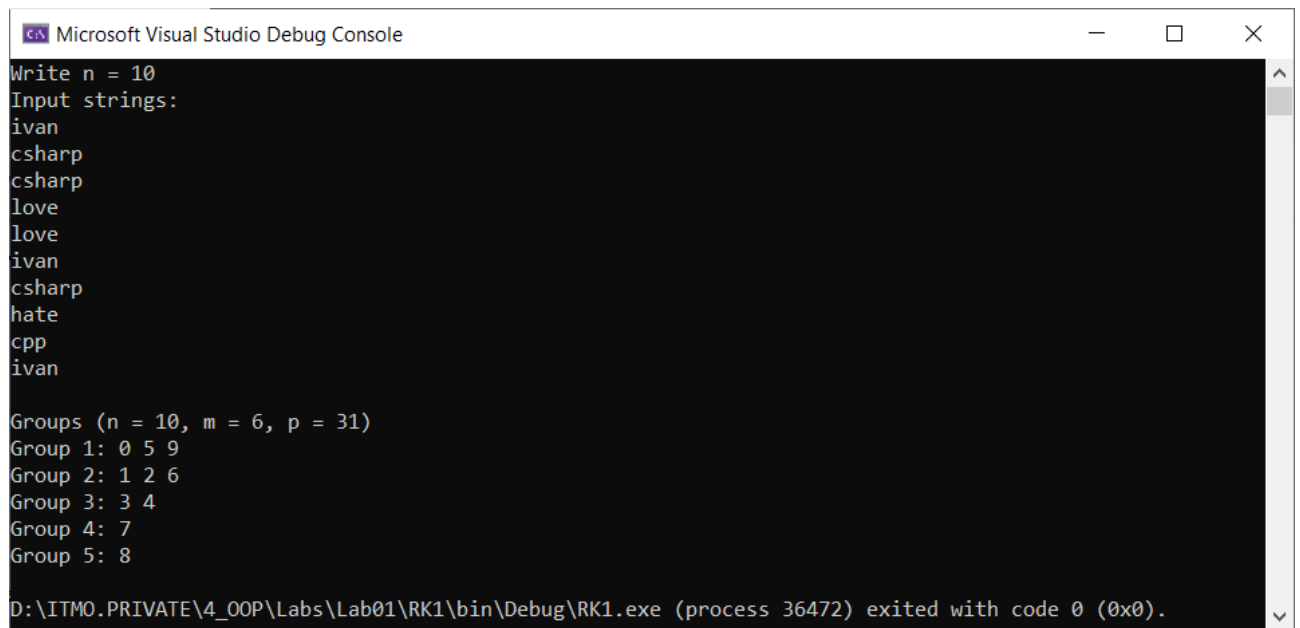
Рисунок 1 — Ввод строк и предрасчёт степеней

Далее слова распределяются по группам в словаре в зависимости от его хэша и выводятся на экран (рисунок 2).

```
1      // Collect groups
2      Dictionary<int, List<int>> groups = new Dictionary<int,
3      ↪ List<int>>>();
4      for (int i = 0; i < n; i++)
5      {
6          int hash = 0;
7          for (int j = 0; j < strings[i].Length; j++)
8          {
9              hash += (strings[i][j] - 'a' + 1) * p_pow[j];
10
11          if (!groups.ContainsKey(hash)) groups[hash] = new List<int>();
12
13          groups[hash].Add(i);
14      }
15
16      // Output
17      int n_group = 1;
18      Console.WriteLine("\nGroups (n = {0}, m = {1}, p = {2})", n, m,
19      ↪ p);
20      foreach (var group in groups)
21      {
22          Console.WriteLine(
23              "Group {0}: {1}",
24              n_group++,
25              string.Join(" ", group.Value)
26          );
27      }
```

Рисунок 2 — Составление и вывод групп

На рисунке 3 представлен результат работы программы.



The screenshot shows the Microsoft Visual Studio Debug Console window. The title bar reads "Microsoft Visual Studio Debug Console". The console output is as follows:

```
Write n = 10
Input strings:
ivan
csharp
csharp
love
love
ivan
csharp
hate
cpp
ivan

Groups (n = 10, m = 6, p = 31)
Group 1: 0 5 9
Group 2: 1 2 6
Group 3: 3 4
Group 4: 7
Group 5: 8

D:\ITMO.PRIVATE\4_OOP\Labs\Lab01\RK1\bin\Debug\RK1.exe (process 36472) exited with code 0 (0x0).
```

Рисунок 3 — Проверка работы программы

2 УПРАЖНЕНИЕ 2

Был написан ввод строк, предварительный расчёт степеней, расчёт хеша подстроки и расчёт хешей префиксов текста (рисунок 4).

```
1      static void Main(string[] args)
2      {
3          // Input
4          int p = 31;
5          Console.Write("s = ");
6          string s = Console.ReadLine();
7          Console.Write("t = ");
8          string t = Console.ReadLine();
9          int n = s.Length;
10         int m = t.Length;
11
12         // Calculate powers
13         int[] p_pow = new int[m];
14         p_pow[0] = 1;
15         for (int i = 1; i < m; i++)
16         {
17             p_pow[i] += p_pow[i - 1] * p;
18         }
19
20         // Hash of s
21         int s_hash = 0;
22         for (int i = 0; i < n; i++)
23         {
24             s_hash += (s[i] - 'a' + 1) * p_pow[i];
25         }
26
27         // Prefix hashes of t
28         int[] t_hashes = new int[m];
29         for (int i = 0; i < m; i++)
30         {
31             t_hashes[i] += (t[i] - 'a' + 1) * p_pow[i];
32             if (i != 0) t_hashes[i] += t_hashes[i - 1];
33         }
```

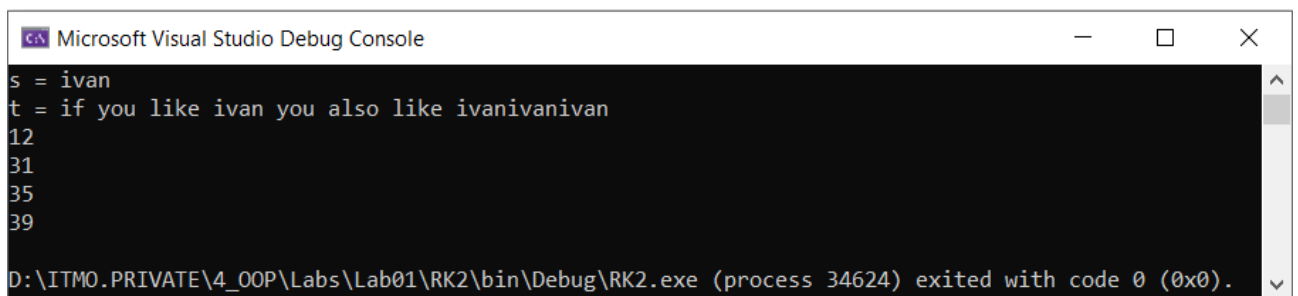
Рисунок 4 — Ввод строк, расчёт степеней и хешей

По найденным префиксным хешам были найдены все вхождения подстроки (рисунок 5).

```
1      // Find s in t
2      int current_hash = 0;
3      for (int i = 0; (i + n - 1) < m; i++)
4      {
5          current_hash = t_hashes[i + n - 1];
6          if (i != 0) current_hash -= t_hashes[i - 1];
7          if (current_hash == s_hash * p_pow[i])
8          {
9              Console.WriteLine(i);
10         }
11     }
12
13
14 }
```

Рисунок 5 — Поиск и вывод вхождений подстроки

На рисунке 6 представлен результат работы программы.



```
Microsoft Visual Studio Debug Console
s = ivan
t = if you like ivan you also like ivanivanivan
12
31
35
39
D:\ITMO.PRIVATE\4_OOP\Labs\Lab01\RK2\bin\Debug\RK2.exe (process 34624) exited with code 0 (0x0).
```

Рисунок 6 — Проверка работы программы

3 КОНТРОЛЬНЫЕ ВОПРОСЫ

- 1) **Какие задачи решаются с помощью алгоритма хеширования?**
 - Быстрый поиск, вставка и удаление в структурах данных (словари, множества).
 - Проверка целостности данных (контрольные суммы).
 - Хранение паролей (криптографическое хеширование).
 - Поиск дубликатов (как в задаче с одинаковыми строками).
 - Разделение данных на группы (равномерное распределение).
- 2) **Как решается проблема коллизий при хешировании?**
 - Идеальное хеширование: подбор хеш-функции, исключающей коллизии для данного набора ключей.
 - Метод цепочек (chaining): в каждой ячейке хранится список элементов, попавших в один хеш.
 - В C# Dictionary<TKey,TValue> использует цепочки (массив + связанные списки/массивы).
- 3) **Какие ограничения и условия на применение алгоритма хеширования?**
 - Хеш-функция должна быть детерминированной и быстрой.
 - Желательно равномерное распределение ключей.
 - Размер таблицы обычно выбирают простым числом или степенью двойки.
 - Для криптографических задач требуется необратимость и устойчивость к коллизиям.
- 4) **Какие конструкции ООП C# применяются для реализации алгоритма хеширования?**
 - Коллекция Dictionary<TKey,TValue> — используют хеширование внутри.
 - Структура KeyValuePair<TKey,TValue> для хранения пар.
- 5) **Как оценивается сложность алгоритма хеширования?**
 - $O(nm)$ для алгоритма группировки
 - $O(n)$ для алгоритма Рабина-Карпа
- 6) **Как можно оценить точность алгоритма хеширования? Вероятность коллизии.**

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы изучены и реализованы средствами ООП С# алгоритмы хеширования. Были написаны и протестированы два алгоритма: группирование строк и поиск подстроки.